MATHSENSEI: A Tool-Augmented Large Language Model for Mathematical Reasoning

Debrup Das*[1], Debopriyo Banerjee[2], Somak Aditya[1], and Ashish Kulkarni[2]

[1]IIT Kharagpur
debrupdas@iitkgp.ac.in, saditya@cse.iitkgp.ac.in
[2]Rakuten Group, Inc. Rakuten Institute of Technology India
{debopriyo.banerjee,ashish.kulkarni}@rakuten.com

Abstract

Tool-augmented Large Language Models (*TALM*s) are known to enhance the skillset of large language models (LLMs), thereby, leading to their improved reasoning abilities across many tasks. While, *TALM*s have been successfully employed in different question-answering benchmarks, their efficacy on complex mathematical reasoning benchmarks, and the potential complementary benefits offered by tools for knowledge retrieval and mathematical equation solving are open research questions. In this work, we present MATHSENSEI, a tool-augmented large language model for mathematical reasoning. We study the complementary benefits of the tools - knowledge retriever (Bing Web Search), program generator + executor (Python), and symbolic equation solver (WolframAlpha-API) through evaluations on mathematical reasoning datasets. We perform exhaustive ablations on MATH, a popular dataset for evaluating mathematical reasoning on diverse mathematical disciplines. We also conduct experiments involving well-known tool planners to study the impact of tool sequencing on the model performance. MATHSENSEI *achieves 13.5% better accuracy over* `gpt-3.5-turbo` *with Chain-of-Thought on the MATH dataset. We further observe that TALMs are not as effective for simpler math word problems (in GSM-8K), and the benefit increases as the complexity and required knowledge increases (progressively over AQuA, MMLU-Math, and higher level complex questions in MATH).* The code and data are available at https://github.com/Debrup-61/MathSensei

1 Introduction

State-of-the-art Large language models (LLMs), including gpt-3.5-turbo, GPT-4, and open-source counterparts, such as Llama 2 have demonstrated impressive performance across a broad spectrum of NLP tasks (Brown et al., 2020; Radford et al., 2019; Chowdhery et al., 2024; OpenAI, 2023). However, their consistent failure on established reasoning dimensions, such as mathematical, commonsense, abductive, and multi-hop reasoning (Lu et al., 2023b; Cobbe et al., 2021; Huang and Chang, 2023) have led the research community to explore various solutions for enhancing their reasoning abilities. This pursuit has given rise to techniques, such as - (1) **intelligent prompting variations**, such as chain of thought (Wei et al., 2022), program of thought (Chen et al., 2023c), tree of thoughts (Yao et al., 2023), and self-refinement (Madaan et al., 2023), (2) **program-guided solving** that generates python code as intermediate steps and offloads execution to a symbolic interpreter (Gao et al., 2023), (3) **multi-model interaction frameworks**, such as Multi-agent Debate (Du et al., 2023; Liang et al., 2023) and Round-Table Conference (Chen et al., 2023b), 4) **tool-augmented LLMs** powered by external symbolic tools, APIs, and libraries (Schick et al., 2023; Lu et al., 2023a; Paranjape et al., 2023; Yang and Narasimhan, 2023; Xie et al., 2023).

In this work, we study the effectiveness of tool-augmented LLMs (*TALM*) applied to problems involving mathematical reasoning. Recent advancements in *TALM* frameworks, such as Chameleon (Lu et al., 2023a), OlaGPT (Xie et al., 2023), ART (Paranjape et al., 2023), and SocraticAI (Yang and Narasimhan, 2023) have explored the effectiveness of incorporating external tools for solving knowledge-intensive reasoning tasks and fundamental mathematical problems (such as, arithmetic and algebra). However, the effectiveness of *TALM* framework is yet to be validated on mathematical reasoning tasks involving complex computations. In this context, it is imperative to assess the suitability of specific tool combinations across diverse mathematical domains (e.g., PreAlgebra, Calculus, Geometry, Intermediate Algebra, Probability) at varying levels of difficulty. This motivated us to undertake a thorough evaluation of *TALM* framework in the context of complex mathematical reasoning tasks. We propose and develop MATHSENSEI, a *TALM*-based framework, comprising a distinct set of tools (also referred to as modules), combined in a sequential fashion. These modules include LLM-based components, such as - knowledge retriever (KR), python code generator (PG), code refiner (CR), , and solution generator (SG); and APIs, such as - Bing-Web-Search-API (BS) and WolframAlpha-API (WA). As illustrated in Fig. 1, MATHSENSEI adopts the modular architecture

*Work done during internship at Rakuten India Enterprise Private Limited.

Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 942–966
June 16-21, 2024

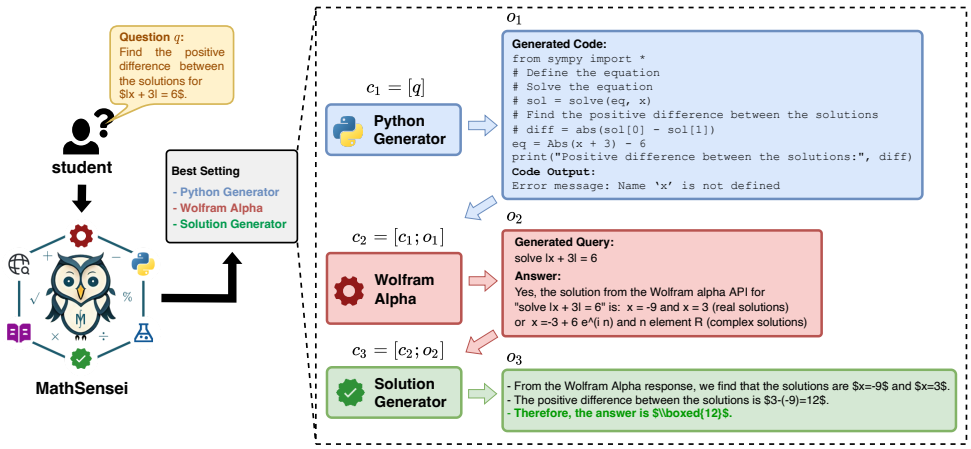

Figure 1: **An end-to-end workflow of MATHSENSEI on the compositional setting from the MATH dataset. The final answer is higlighted in green font.**

.

TALM	Math Discipline	Form	Search	Python	WAlpha	Plan	Tool-Study
OlaGPT	Algebra	MCQ	✓	✗	✗	Plan-And-Solve	✗
Chameleon	✗	✗	✓	✓	✗	Plan-And-Solve	✗
ART	Algebra	Open	✓	✓	✗	Call-as-req	✗
MATHSENSEI	Algebra, Precalculus, Geometry, Probability, Number Theory & more	Both	✓	✓	✓	Both	✓

Table 1: Comparison of MATHSENSEI with state-of-the-art Tool-Augmented LLMs; **Form** - Question-Answer Format (MCQ with multiple options, Open/Subjective), **Search** - Use of Web Search, **Python** - Python code guided problem solving, **WAlpha** - WolframAlpha, **Tool-Study** - Study of each tool, **Plan** - Planning Strategy used; **Plan-And-Solve** - Determine the sequence of tools to be executed beforehand, **Call-as-req** - Dynamically decide to call tool when required at a step during execution.

from Chameleon (Lu et al., 2023a). Through systematic experiments of MATHSENSEI, we aim to discern the effectiveness of each module in addressing specific types of mathematical problems, having varying levels of difficulty.

Our extensive ablations (varying the set and order of modules), show that complex mathematical problems, spanning different subdomains can be benefited by specific types, combinations, and *order* of the modules. This further highlights the need for planning strategies. We evaluate two advanced planning techniques within our pipeline, investigating methodologies such as Plan-And-Solve (Lu et al., 2023a) and REACT (Yao et al., 2022) with MATHSENSEI.

We make the following contributions:

1. We comprehensively evaluate the effectiveness of *TALM* frameworks across multiple mathematical datasets, such as GSM-8K, AQUA-RAT, MATH, MMLU-Math, encompassing diverse mathematical problem types and tasks. Compared to MATH, MMLU-Math, our experiments on simpler mathematical datasets (e.g., GSM-8K, AQUA-RAT) reveal minimal benefit of using multiple modules on top of CoT prompting.

2. Through systematic ablations by varying the set and order of modules in our framework, we observe that complex mathematical problems spanning different domains (such as, algebra, calculus, number theory, and probability from the MATH dataset) can be benefited by certain types, combinations, and *order* of these modules. We observe that the BS module outperforms the KR module for retrieving relevant knowledge for mathematical problems. The setting of WA+BS+SG outperforms PG+SG, demonstrating that program-guided solving techniques (Gao et al., 2023; Drori et al., 2022) may not be universally suitable for all mathematical problems. These findings motivate the necessity of exploiting better planning techniques.

Our best configuration of MATHSENSEI, PG+WA+SG achieves an impressive performance accuracy of 47.6 % on the MATH dataset, surpassing `gpt-3.5-turbo`() with Chain-of-Thought (CoT) prompting by 13.5% (Chen et al., 2023a). The same setting shows a performance gain of +11.6% over GPT-4 (with CoT prompting) on Intermediate Algebra problems. For Precalculus, GPT-4 (with CoT prompting) has an accuracy of 26.7%, which gets improved to 28.9% by our WA+PG+SG setting. Improvements on AQuA-RAT and MMLU-Math are lower, 2.4% and 3.3% respectively, showing the efficacy decreases as requirement of external knowledge decreases.

3. We quantify the performance of state-of-the-art planning techniques, such as Plan-And-Solve and REACT coupled with tool-augmented LLMs on the MATH dataset. However, we do not observe benefit of using the planners over our best configurations of PG+WA+SG, which may indicate a need for developing targeted planning strategies for mathematical *TALMs*. We include our Planning related experiments in the Appendix.

2 Related Work

Prompting Techniques. Large Language Models (LLMs) employing prompting strategies such as Chain-of-Thought (CoT) (Wei et al., 2022) and Program-of-Thought (POT) (Chen et al., 2023c) have demonstrated commendable performance on simple mathematical datasets such as GSM-8K (Cobbe et al., 2021). However, their efficacy diminishes for datasets requiring complex computations and advanced mathematical knowledge. For instance, on the MATH dataset, GPT-4 exhibits a notably low accuracy of around 50%. Several variations of these strategies have been explored to improve accuracy in reasoning tasks. (Madaan et al., 2023) proposed *self-refine* that involves iteratively refining the initial output by utilizing feedback from the same model. (Zhou et al., 2024) employs code-based self-verification, by utilizing python code to check simple constraints that the LLM generated output should satisfy and correcting the output if necessary. Similarly, Progressive-Hint-Prompting (Zheng et al., 2023) involves multiple turns of interactions, using previously generated answers as hints for subsequent turns. Similar to POT prompting, PAL (Program Aided language models) (Gao et al., 2023) adopts a program-guided solving paradigm. It reads natural language problems, generates programs as intermediate reasoning steps, and delegates the solution step to a runtime environment, such as a Python interpreter. Across 13 natural language reasoning tasks within Big-Bench-Hard (Suzgun et al., 2023), they observe that program-guided solving consistently outperforms significantly larger models.

In our Tool-augmented framework (MATHSENSEI), we incorporate several such techniques. We adopt CoT prompting for the text generation modules, and use the methodology by (Gao et al., 2023) to generate python code (using libraries like `sympy`) based on the current context and mathematical question; followed by execution of the code using python interpreter. While (Gao et al., 2023) focuses on elementary level MWP (Math Word problems) and simple arithmetic datasets such as ASDIV (Miao et al., 2020) and SingleEQ (Koncel-Kedziorski et al., 2015), we explore complex mathematical datasets spanning diverse math problem types (MATH, AQUA (Ling et al., 2017), MMLU-Math). Following *self-refine*, we employ a code refinement module to iteratively rectify syntactical errors in the original generated code, using error messages from the interpreter.

Tool-Augmented LLMs. The emerging trend of tool-augmented LLMs has garnered increasing attention within the research community. Large language models, trained on the objective of next-token prediction, excel at generating tokens based on probabilistic patterns in their training data, making them effective in data-intensive tasks. However, their proficiency falls short in capturing nuanced reasoning or token relationships, particularly in mathematical domains. Consequently, there are instances or specific question types where it would be advantageous for an LLM to leverage support from specialized tools or modules. For instance, consider a question requiring the solution to the roots of a 4th-degree polynomial. The LLM, upon generating a special token followed by a query, can pause its generation and invoke a mathematics computing platform WolframAlpha. WolframAlpha, in turn, can utilize its API to process the query and return the answer to the LLM, which can then continue its generation. Toolformer (Schick et al., 2023) leverages data annotated with such tool calls (using special tokens for tools) and responses to train language models to employ tools as needed in a self-supervised manner. Similarly, the tool-augmented LLM framework CHAMELEON (Lu et al., 2023a) adopts a plug-and-play approach to utilize tools sequentially. In their setup, the sequence of execution of the tools is predetermined based on a target task; the output of each tool is added to the context for subsequent downstream tools in the pipeline. They perform evaluation on multi-modal knowledge-intensive datasets, such as ScienceQA and TabMWP. Similarly, frameworks such as ART (Paranjape et al., 2023) engage in multi-step reasoning, where each step is linked to a tool call. Utilizing search and code tools, ART tackles various tasks across datasets such as MMLU (Hendrycks et al., 2021a)and BigBench (Srivastava et al., 2023).

Our work adopts the generic backbone of popular tool-augmented LLM frameworks such as Toolformer and CHAMELEON. In comparison to the previous work, we distinguish ourselves by conducting a comprehensive analysis and comparison specific to tools useful for addressing diverse mathematical problems. Notably, CHAMELEON lacks evaluation on mathematical datasets, and ART focuses exclusively on algebra, leading to gaps in the assessment of tool-augmented LLMs. Furthermore, our study incorporates a comparison of planning techniques within tool-augmented LLM frameworks for mathematical reasoning, an aspect not adequately addressed in the current literature. To the best of our knowledge, planning techniques like REACT (Yao et al., 2022) have primarily been tested on knowledge-intensive reasoning datasets such as FEVER (Thorne et al., 2018) and HotpotQA (Yang et al., 2018).

3 Methodology

We first discuss some notations to formalize the problem. Let M denote the set of modules[1] (each performing a specific task), p_i be the input prompt for module m_i, and Q be the set of mathematical queries.

3.1 Problem Formulation

Given an input mathematical query $q \in Q$, the objective is to provide the final correct answer a by executing

[1]The modules can be viewed as external tools, where each module $m \in M$ can be either powered by LLMs, such as Python code generators, Knowledge Retrievers, or they can be non-LLM API tools, such as WolframAlpha, Bing Web Search.

the set of relevant modules. Let $[m_1, \ldots, m_t]$, be the ordered sequence of chosen modules for answering q, and $[o_1, \ldots, o_t]$ be the output sequence of the t modules. Let, s_i, f_i, and c_i denote the instruction, in-context example(s), and context, respectively, that we use for module m_i. The input prompt p_i, corresponding to module m_i is defined as:

$$p_i = \langle s_i; f_i; c_i \rangle \tag{1}$$

where context c_i is defined as:

$$c_i = \begin{cases} [q], \text{ if } i = 1; \\ [c_{i-1}; o_{i-1}], \text{ for } i = 2, \ldots, t \end{cases} \tag{2}$$

Here, $x; y$ denotes concatenation of x and y.

3.2 Modules

In this section, we present a brief overview of the tools or modules that we use in our study. We show the list of model/api used for each module in Table 8. A detailed description of the prompts used in each module is presented in the Appendix section.

• **LLM-based Knowledge Retrieval (KR) -** For this module, we design a prompt to extract relevant knowledge from a pre-trained LLM (taking any one from the list of models mentioned in Table 8) in the form of concepts, formulas, mathematical expressions, theorems, definitions, and hints on how to solve a corresponding mathematical question. An example prompt and output is shown in Table 19 in Appendix.

• **Bing Web Search (BS) -** This module queries the `Bing-Web-Search-API` (🌐) to extract the most relevant snippets which may contain similar questions and concepts required for solving a mathematical problem. For similar questions search, we directly query the `API` with the mathematical question. In case of concepts search, we first use an LLM (either gpt-3.5-turbo or text-davinci-003) to generate a query corresponding to the input question, and then call the `API` to retrieve relevant concepts (refer to Fig. 2 for an example).

• **WolframAlpha (WA) -** This module (comprising multiple components) calls the `WolframAlpha-API` using a query in the Wolfram language, retrieving the mathematical information from this knowledge base and utilizing the capabilities of its computation engine. First we employ an LLM to generate contextualized thoughts. Subsequently, based on the generated thought, the next component formulates a Wolfram code language query (referred to as the *"Final Query"*). On passing this query as input to the WolframAlpha-API, we get a `JSON` dictionary object. We extract all the useful information from this dictionary (using an LLM-based extractor) and add it to the context of next module. An overview of the **WA** module is presented in Fig. 3.

• **Python Generator+Executor (PG) -** We use an LLM that takes as input the current context as a part of a well-structured prompt (shown in Fig. 4). The LLM is explicitly instructed to use the `sympy` library for accessing a set of mathematical operations and data structures

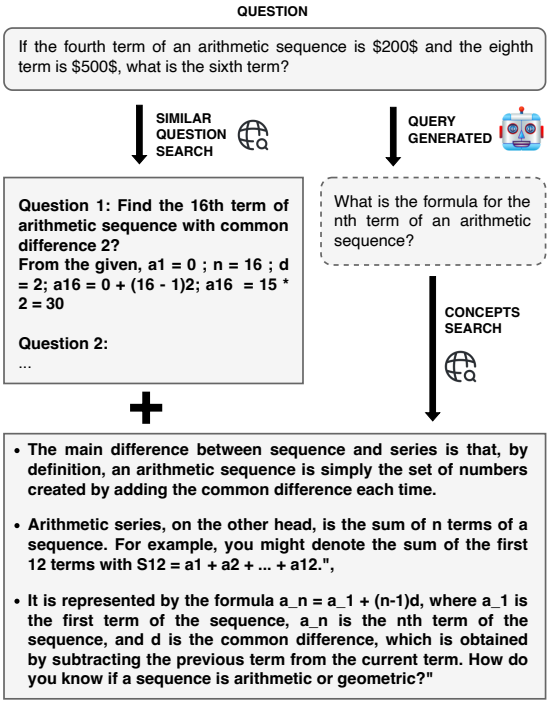

Figure 2: Overview of the BS module; We concatenate the similar questions and concepts (which is then used by a downstream module).

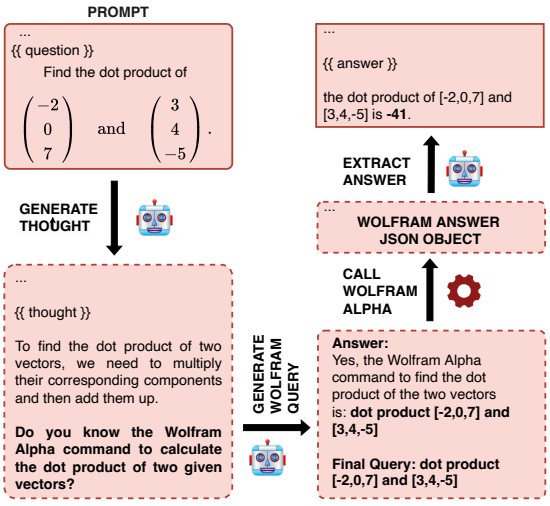

Figure 3: Overview of the WA⚙ module.

required. Based on the prompt, the module generates an (executable) Python code, which on execution returns some output(s) or an error message. We handle syntax errors using two setups:

- **Without refinement**: Here, if generated code produces syntax errors, we omit the output of PG from the context for next module.
- **Code-Refinement** (CR): Here, we feed the error message along with the incorrect program to a code-fixing LLM which then generates a corrected python code and rationales of fixed errors given

as "*Errors fixed*". We also add the information of common errors from our qualitative analysis in the system prompt to aid the code refinement process. An output for the code refinement setup from the MATH dataset is presented in Fig. 4 (Appendix).

• **Solution Generator (SG) -** The solution generator is the final module in all settings. It takes the output from the pipeline and compiles a step-by-step solution based on all the context of previous modules. The final step is prompted to produce the answer of the question. It outputs the final answer enclosed within $\\$\\$boxed{}$\$ for the MATH dataset.

4 Experimental Setup

We first introduce the mathematical datasets used in our study (§4.1), followed by the experiments that we perform with various combinations of modules (§4.2). We use `gpt-3.5-turbo` as the default LLM in LLM-based modules unless mentioned otherwise. This is mainly because it is more accessible and cheaper compared to GPT-4. For querying a search-engine, we use `Bing-Web-Search-API`. Please refer to 23 in Appendix for details about online resources that we use.

4.1 Datasets

MATH. The MATH dataset (Hendrycks et al., 2021b) serves as the primary dataset for our work. It covers *5000* mathematical problems, which are categorized into seven subject types (*Precalculus, Prealgebra, Algebra, Geometry, Intermediate Algebra, Counting and Probability, and Number Theory*) and *five levels* of difficulty (ranging from 1 to 5, where 1 denotes the least difficult and 5 denotes the most difficult). Our choice of the MATH dataset is motivated by its unique characteristics: Unlike many datasets, scaling up LLMs (in terms of model parameters) does not necessarily enhance accuracy on MATH. The dataset also poses intricate challenges, going beyond simple arithmetic or high school mathematics problems.

AQUA-RAT. The AQUA-RAT dataset (Ling et al., 2017) contains *253 algebraic math word problems* with rationales. Unlike the MATH datset, it has a *multiple-choice* answer format with five options. It allows us to evaluate MATHSENSEI on mathematical problems in the domain of algebra.

GSM-8K. GSM-8K (Cobbe et al., 2021) contains *high school level math word problems* which require basic arithmetic operations (addition, subtraction, multiplication, and division) to reach the final answer. The final answer is always an integer value. We use all 1319 examples from GSM-8K test set for evaluation.

MMLU-Math. The MMLU dataset (Hendrycks et al., 2021a) covers *57* diverse tasks (including *elementary mathematics*, *US history*, *computer science*, *etc.*), which require extensive problem solving abilities and world knowledge. For this work, we use the mathematical test subset of MMLU, known as MMLU-Math that contains *974* mathematical questions spanning 5 types - *abstract algebra, elementary mathematics, high-school mathematics, college mathematics*, and *formal logic*. Similar to AQUA-RAT, MMLU-Math also has a *multiple-choice* answer format.

4.2 Experiments

We conduct several experiments by meticulous analysis of individual modules **in the domain of complex mathematical reasoning**, through systematic ablations on the module sequences. For some of our ablations, we use different variants of OpenAI models, such as text-davinci-002 and text-davinci-003 other than the default gpt-3.5-turbo. We also employ models from the Llama family, such as Llama-2-7B and Phind-Code-Llama-34B-V2. We use accuracy as our evaluation metric for comparing different settings. Our experiments enquire the following questions:

• What is the impact of adding LLM generated mathematical knowledge relevant to the question **[KR module]** before invoking the Solution Generator module **[SG module]**? (§5.1)

• How does Bing Web Search **[BS module]** compare against the LLM-based knowledge generation **[KR module]** for the task of adding relevant mathematical knowledge and information to the problem solving process? (§5.1, §5.2)

• What is the utility of augmenting mathematical knowledge-bases, such as WolframAlpha **[WA module]** with LLMs for solving problems across different levels of complexity? How does it compare against the paradigm of program-guided solving? (§5.3)

• What are the benefits of using program-guided complex problem solving **[PG module]**, and impact of LLM-based code refinement **[CR module]** in case of syntactical errors? (§5.4)

• What is the effect of using **multiple modules together**? How does the benefit vary with the **difficulty level, mathematical subject type, and dataset?** (§5.5)

• How to plan effective utilization of these modules? How does non-adaptive planning strategies **[Plan-And-Solve]** compare against dynamic planning strategies such as **[REACT]** which uses a thought, action, and observation based mechanism. (Appendix A)

5 Effects of Adding Modules over LLMs

Here, we present results and analyze the impact of adding individual modules on top of the original LLM CoT variant (termed SG): KR in §5.1, BS in §5.2, PG in §5.4, and WA in §5.3. For each module, we also provide ablations over different LLMs (as applicable).

5.1 LLM-Based Knowledge Retrieval (KR)

Recently, Chameleon (Lu et al., 2023a) demonstrated an accuracy boost for knowledge intensive QA datasets, such as ScienceQA and TabMWP by using the KR module. Skills-In-Context prompting (Chen et al., 2023a)

Model	Ovr Acc
text-davinci-002 (🤖)	22.8
text-davinci-003 (🤖)	27.1
Llama2-7B (🦙)	28.4
gpt-3.5-turbo (🤖)	34.4

Table 2: Performance of different backbone models used for KR module in the KR+SG setting. For all settings, we use `gpt-3.5-turbo` as the default LLM for the SG module.

also shows similar results by utilizing some basic skills (such as mathematical theorems) during generation. Following the literature, we investigate the impact of adding relevant knowledge (such as mathematical concepts and formulae) using an LLM-based KR module in the context of SG module, and examine the efficacy of the KR+SG setting on the MATH dataset (Table 4). We also ablate over different LLMs (Table 2) to power the KR module, while fixing the SG module to `gpt-3.5-turbo`.

Results. As shown in of Table 4, the extra knowledge retrieved by the KR module is useful only for problems in Algebra, PreAlgebra, and Probability domains. Moreover, the overall accuracy drops steadily as we change **KR's** LLM from `gpt-3.5-turbo` to other variants (shown in Table 2). This indicates that, generic LLMs (such as those mentioned in Table 2) are not equipped with mathematical concepts of other domains (Precalculus, Geometry, Number Theory, Intermediate Algebra). After analyzing different LLM variants for the KR module, we find that the knowledge retrieved by weaker LLMs heavily degrades performance of the downstream SG module. This motivated us to explore the impact of search engine-based knowledge retrieval (detailed in §5.2).

5.2 Bing Web Search (BS)

We investigate the advantages of adding a search engine-based knowledge retrieval module (BS) as an alternative of KR for **similar questions search** and **concepts search** before applying SG.

Results. In Table 3, we observe that BS+SG setting is a clear winner over the SG setting, when `gpt-3.5-turbo` is used for generating the `Bing-Web-Search-API` query and getting final solution from SG. This holds true even if the stand-alone SG is varied between `text-davinci-003` ($+22.5\%$) and `gpt-3.5-turbo` ($+4.2\%$). Thus, augmenting LLMs with knowledge (relevant to a mathematical question) retrieved from the web proves to be beneficial in improving problem solving capabilities. The use of `text-davinci-003` alone or in combination with `gpt-3.5-turbo` for BS and SG modules, diminishes the performance of both BS+SG and SG settings, which is expected (Ye et al., 2023).

	Setting		
LLMs	(🌐+✅)	(⚙️+✅)	(✅)
(🤖+🤖)	38.7	42.6	-
(🤖+🤖)	27.4	35.6	-
(🤖+🤖)	30.0	37.8	-
(🤖+🤖)	20.8	27.0	-
(🤖)	-	-	34.5
(🤖)	-	-	16.2

Table 3: Ablations of BS+SG (🌐+✅), WA+SG (⚙️+✅), and SG (✅) settings using different combination of LLMs, such as `gpt-3.5-turbo` (🤖) and `text-davinci-003` (🤖) on the MATH dataset.

5.3 WolframAlpha Search (WA)

We compare the performance of WA+SG and SG settings on the MATH dataset in Table 3. We perform ablations with text-davinci-003 and gpt-3.5-turbo as the LLMs used in WA for query generation and answer extraction.

Results. From Table 3, we observe that WA+SG outperforms the SG approach by 8.1%, when both WA and SG are powered by `gpt-3.5-turbo`. This shows a clear and significant contribution of complementary strengths coming from the knowledge retrieved through WolframAlpha. Furthermore, it is notable that the observed benefits of the WA module cannot be solely attributed to the characteristics of the LLMs employed for query generation or answer extraction. This is evident from the substantial performance gains (around 10.8%) achieved, even after enabling both WA and SG with a comparatively weaker model, such as `text-davinci-003`. Additionally, the mix of `text-davinci-003` and `gpt-3.5-turbo` for the WA+SG setting demonstrates superior performance compared to SG with `gpt-3.5-turbo`, achieving improvements of 1.1% and 3.3%, respectively. Thus, showcasing meaningful positive impact of augmenting WA with the stand-alone SG module.

5.4 Python Generator (PG)

In this section, we investigate the effectiveness of the Python Generator (PG) module in using python code, and an interpreter to solve mathematical problems (utilizing external symbolic libraries from `Sympy`). Following, PAL (Program Aided Language Models) (Gao et al., 2023), Program of thought (Chen et al., 2023c), our PG module consists of a a program generator and an executor. The generated code and corresponding output are added in context of the next module in sequence. We present the results of the PG+SG setting in Table 4 for the MATH dataset. For MATH, we present three variations: (i) PG+SG with no code refinement, (ii) PG+CR+SG with code refinement, and (iii) PG′[🎵]+SG (where PG′[🎵] denotes the use of Phind-CodeLLama-34B-V2 model for PG. We choose **Phind-CodeLLama-34B-V2** for our

Method	Alg	P.Cal	P.Alg	Geom	Prob	N.Th	Int.Alg	O.Acc
Baselines with `gpt-3.5-turbo` (🤖)								
CoT-LTP (Guo et al., 2023)	49.6	16.3	52.3	22.5	30.2	29.8	16.9	31.1
ComplexCoT (Fu et al., 2023)	49.1	16.8	53.8	22.3	29.7	33.4	14.6	34.1
ComplexCoT+PHP (Zheng et al., 2023)	51.1	16.1	57.7	25.4	33.7	35.1	17.1	36.5
SKiC (Chen et al., 2023a)	57.9	23.0	62.0	30.1	38.2	35.5	17.8	40.6
Baselines with `GPT-4`								
CoT (Zhou et al., 2024)	70.8	26.7	71.6	36.5	53.1	49.6	23.4	50.4
PHP (Zhou et al., 2024)	74.3	29.8	73.8	41.9	56.3	55.7	26.3	53.9
Ours								
SG (✅)	46.7	18.1	55.7	25.3	32.9	30.2	16.2	34.5
KR+SG (📖+✅)	49.1	15.0	58.0	24.4	34.3	29.6	12.0	34.4
BS+SG (🌐+✅)	51.6	20.1	63.3	27.1	36.1	39.6	16.3	38.7
PG+SG (🐍+✅)	60.0	26.5	66.1	30.7	42.1	40.5	21.1	44.6
PG+CR+SG (🐍+🧪+✅)	59.7	25.2	63.9	26.9	48.3	43.0	26.9	44.8
PG′[🔏]+SG (🐍+✅)	55.4	23.5	58.0	22.9	32.7	42.2	17.9	39.6
WA+SG (⚙️+✅)	57.8	26.1	58.5	26.3	37.6	37.8	31.5	42.6
PG+BS+SG (🐍+🌐+✅)	53.1	20.7	58.7	28.6	37.8	36.6	19.9	39.0
BS+PG+SG (🌐+🐍+✅)	55.0	23.1	61.2	27.5	35.4	35.4	20.5	39.8
WA+PG+SG (⚙️+🐍+✅)	62.5	28.9	61.5	27.1	42.6	45.7	33.4	46.3
PG+WA+SG (🐍+⚙️+✅)	61.6	28.7	64.7	30.5	42.8	49.1	35.0	47.6
BS+WA+SG (🌐+⚙️+✅)	56.2	22.9	61.0	29.8	37.5	44.0	28.9	42.9
WA+BS+SG (⚙️+🌐+✅)	60.0	27.0	65.0	29.0	40.5	42.2	31.4	45.4
BS+PG+WA+SG (🌐+🐍+⚙️+✅)	60.2	26.4	65.0	31.3	44.7	48.7	31.6	46.7

Table 4: Comparison of our Modular Settings to Published Baselines on **MATH**. We use `gpt-3.5-turbo` (🤖) as the default LLM for each setting (except one row). For PG′[🔏]+SG (🐍+✅) setting, we use `Phind-CodeLlama-34B-V2` as the underlying LLM for the PG🐍 module (while keeping `gpt-3.5-turbo` (🤖) as the default LLM for SG✅ module); Alg: Algebra, P.Cal: Precalculus, P.Alg: Prealgebra, Geom: Geometry, Prob: Probability, N.Th: Number Theory, Int.Alg: Intermediate Algebra; We have taken the first four baseline results from SKiC (Chen et al., 2023a), and following two baselines from (Zhou et al., 2024).

ablation since it is the best model from the huggingface Code-LLM leaderboards. The Phind family of models are finetuned versions of CodeLlama-34B on a Phind dataset consisting of 80k high quality programming problems and solutions.

Results In Table 4, we observe that the PG+SG setting using the `sympy` library without code refinement can improve upon the performance accuracy of SG on the MATH dataset by a margin of 10.1%. We find that a majority of problems in MATH require complex computations such as solving equations, representation of complex mathematical objects such as vectors, solving problems in Geometry, some of which are hurdles for the Solution generator module since text representations alone fail to capture such complexities. Libraries such as `Sympy`, on the other hand, has support for symbolically representing such objects using well defined functions, classes, methods, and sub-packages. We find that this helps PG outperform SG on all mathematical types in MATH. The outcomes of our experiment with PG+CR+SG setting only yields marginal enhancements on overall accuracy. We also observe a drop in the accuracy by 5% when using Phind-CodeLLama-34B-V2 as the LLM in PG module.

Setting	FL	AA	EM	CM	HM
(✅)	53.9	49.0	84.6	41.0	57.7
(📖+✅)	50.6	43.9	84.8	38.6	58.5
(⚙️+✅)	52.4	54.5	88.1	58.0	67.0
(🌐+✅)	40.5	44.4	80.1	49.0	63.0
(🐍+✅)	49.5	50.0	81.6	44.0	69.4
(⚙️+🐍+✅)	44.7	36.1	81.4	57.1	63.7
(🐍+⚙️+✅)	45.7	55.5	92.1	42.3	68.0
(🐍+🌐+✅)	50.0	47.0	81.2	44.0	59.1
(🌐+🐍+✅)	46.8	38.0	84.9	47.5	63.3
(🌐+🐍+⚙️+✅)	41.3	43.0	79.3	45.0	66.1

Table 5: MMLU Accuracy vs type of problem; FL:Formal logic, AA: Abstract Algebra, EM: Elementary Mathematics, CM: College Mathematics, HM: High School Mathematics

Setting	GSM-8K	AQUA	M.Math
(✅)	77.0	61.4	66.2
(📖+✅)	71.8	57.5	64.5
(⚙️+✅)	61.7	57.9	66.0
(🌐+✅)	56.0	53.5	67.6
(🐍+✅)	74.1	55.1	68.1
(⚙️+🐍+✅)	69.1	63.8	65.1
(🐍+⚙️+✅)	67.6	62.6	67.1
(🐍+🌐+✅)	67.6	58.3	67.2
(🌐+🐍+✅)	69.2	56.3	69.5
(🌐+🐍+⚙️+✅)	70.7	61.4	66.9

Table 6: Comparison of Multi-Module Settings for GSM-8K, AQUA-RAT (AQUA), and MMLU-Math (M.Math) datasets.

5.5 Results of Multiple Module Experiments

We experiment with various module combinations on four datasets MATH, AQUA-RAT, GSM-8K, and MMLU-Math and report in Tabs. 4 & 6. Our findings reveal that distinct modules exhibit specialized efficacy in addressing specific categories of mathematical problems. On the **MATH** dataset, (1) **WA** emerges as a valuable resource for tackling intricate mathematical subdomains, particularly in Intermediate Algebra **(Int.Alg)** and Number Theory **(N.Th)**. The PG+WA+SG setting outperforms SG by 19% on Int.Alg. We conduct a qualitative analysis of PG+SG on 106 randomly sampled questions from MATH spanning all types and difficulty levels, presented in Table 13. We find that the majority of errors in **Int.Alg** arise from python code execution errors and the inability of python code to represent complex math objects in this subdomain. In contrast, the **WA** module effectively interacts with the API using both natural language and symbolic queries (Table 15) to address these issues, resulting in substantial enhancements.(2) For Algebra-related problems (**Prealgebra** and **Algebra**) having complex computations, the generation of Python code guided by PG and the `Sympy` library proves to be an effective choice. The WA+PG+SG setting elevates the performance of SG by 15.8% on Algebra. The PG+SG setting performance is also significantly better compared to SG (10.4%) on Prealgebra showing the utility of code representations over natural language in this subdomain. (3) Table 9 presents an examination of the variations in accuracy among various settings as a function of the problem levels (1-5) in the MATH dataset. Our analysis reveals a consistent improvement of over 10% across all levels with diverse modular configurations. This reaffirms the importance of judiciously selecting tools and configurations based on the specific features and attributes of the given problem.

Effectiveness of MATHSENSEI on MMLU-Math. Results in Table 6 reveal that the BS+PG+SG configuration enhances the accuracy of the SG setting by 3.3%. As the performance is gain is low, we further perform a type wise analysis in Table 5. We observe that, other than **Formal Logic** (FL), adding different modules show substantial improvements in different types, such as 17% in College Math, 11.7% in High School Math, 7.5% in Elementary Math. More specifically we find that: (1) The PG+WA+SG setting improves the accuracy of the SG setting from 84.6% to 92.1% on **Elementary mathematics** problems. (2) Interestingly, problems in **Formal logic** are best solved using SG alone. The drop in performance for the PG+SG setting **(53.9 -> 49.5)** is due to the inability of PG to adequately represent predicate logic, First Order Logic (FOL) sentences through python code, (3) For **College Mathematics**, the WolframAlpha module demonstrates highest efficacy, as evidenced by the substantial benefits observed in both the WA+SG and WA+PG+SG settings. Notably, WA+SG outperforms the SG setting by a significant margin of 17%. Our analysis in MMLU-Math further supports the complementary benefit of the tools used in MATHSENSEI framework for various mathematical types.

Decreased Effectiveness of MATHSENSEI on GSM-8K, and AQUA-RAT. From Table 6, we observe marginal improvements of using multiple modules on **AQUA-RAT** and **GSM-8K**, over the standalone SG module. Both datasets comprise simpler algebraic and arithmetic word problems. GSM-8K consists of problems requiring simple arithmetic operations such as addition, subtraction, etc. and its complexity stems from linguistic diversity. We conduct a case study on a randomly sampled set of 20 examples from **GSM-8K**, where PG+SG is incorrect and SG is correct, we find that 18 (out of 20) have incorrect outputs generated by **PG** (due to reasoning errors) (Table 13). For all these 18 examples, the LLM generated python code tries to solve a simple problem by using complex objects in `Sympy`, which in turn degrades the performance. For the remaining two examples, one has an execution error, while for the other one, **SG** alters the correct PG answer to incorrect. Similar to GSM-8K, **AQUA-RAT** primarily focuses on problems that require generic language-based reasoning skills. We find that settings with tools mostly hurt the performance compared to **SG**. This is attributed to the fact that WA and BS are unnecessary for addressing straightforward problems, and invoking them often introduces noisy and irrelevant information into the context of SG. As we saw previously in case of GSM-8K, a significant proportion of errors in PG+SG (+) can be linked to the application of `Sympy` for simple problems (Table 13). These outcomes highlight the diminishing utility of employing additional modules for tasks requiring minimal external knowledge.

5.6 Insights from Qualitative Analysis of Modules

We consider an example from the MATH dataset and present a qualitative analysis of the responses generated by different settings in Table 7. We observe that SG and PG+SG are unable to capture the fine-grained nuances in the input question (repetition of characters in the word "NINE"), leading to reasoning errors. On the other hand, the BS+SG and WA+SG settings avoid committing such errors. This demands a need for a careful examination of the strengths and limitations of the individual modules, which we discuss in details in this section.

Bing Web Search (BS). Previous investigations of retrieval-augmented generation (RAG) (Lewis et al., 2021) and Self-RAG (Asai et al., 2023) have shown that conditional generation using retrieval-based approaches improves factuality in knowledge intensive tasks such as question answering, fact verification, etc. We observe similar benefits of employing retrieval-based methods in the domain of complex mathematical reasoning. The BS module retrieves useful information (such as formulas, concepts and similar questions) from the Web,

QUESTION: Determine the number of ways to arrange the letters of the word NINE. **GOLD ANSWER:** $\boxed{12}$

SG ( ):

There are 4 distinct letters in the word NINE. The number of ways to arrange 4 distinct letters is 4! = 24. Therefore, the answer is $\boxed{24}$.

SG module misses the repeated characters in NINE.

PG+SG (+)

```
from sympy import *
word = 'NINE'
numperm=factorial(len(word));
print(numperm);
...
```

The word NINE has 4 letters.The number of ways to arrange 4 distinct letters is 4! = 24. Therefore, the answer is $\boxed{24}$.

PG module ignores the repeated characters.

BS+SG (+)

Retrieved Example: In the word "BANANA", there are 6 letters but only 3 distinct letters ...repeated letters.
Retrieved Formula: nPr = n!/(n1! n2! ... nr!) ...Therefore, the answer is 4!/ (2! 1! 1!) = $\boxed{12}$.

BS module retrieves similar examples and relevant formula from the Web.

WA+SG (+)

We call the Permutations function on the word NINE to find all possible arrangements of the letters. The answer from WolframAlpha is EINN, ENIN, ..., NINE. Therefore, the answer is $\boxed{12}$.

WA module generates the correct query: Permutations[NINE] to solve the given question and final SG answer is grounded on WA output.

Table 7: Qualitative Analysis of the Responses generated by different Settings for a given MATH example.

improving the effectiveness of the downstream SG module. As shown in Table 7, the BS module retrieves an informative example of a similar question (permutations of the letters in the word "BANANA", having repeated characters) and the correct formula of permutations with repetitions, which aids the SG module in correctly reasoning about the final solution. However, our current implementation of the BS module also has certain limitations. We directly use the raw output returned by the Bing Web Search API v7, which is **noisy** in certain cases. Additionally, we do not employ any **critique mechanism** to check the relative importance of multiple pieces of the retrieved information. We also observe a significant reduction in performance on GSM-8K after adding the BS module to SG. This calls for a component which can effectively decide when it is required to retrieve knowledge and when it is not necessary (future research).

WolframAlpha (WA). The WA module overcomes the limitations of SG by harnessing the computational power and intelligence of the WolframAlpha engine. In cases, where the query to the WolframAlpha-API is syntactically and logically correct for solving a mathematical question, the returned answer is guaranteed to be correct, which is then processed by the SG module to compile the final answer. From Figure 6, we observe maximum benefit of WolframAlpha module for problems in the subdomains of Algebra and Intermediate Algebra (primarily for difficulty levels greater than one). We demonstrate the utility and limitations of the WA module in Tables 16 and 14, respectively. The limitations of the WA module are mostly associated with: (1) Logical errors in LLM-generated WA API Queries (Example 1 in Table 14), (2) Wrong interpretation of correct WA response by the downstream SG module (Example 2 in Table 14), (3) Single line WA response, which restricts the ability of downstream SG module to generate step-by-step reasoning (Example 3 in Table 14).

Python Generator (PG). As mentioned in Section 5.4, the Sympy library offers strong capabilities to the PG module for **MATH**. The deterministic program executor also helps in avoiding common errors committed by the standalone SG module. We found the PG module to be most useful in solving *Algebra*, *Prealgebra* and *Number Theory* problems. The example presented in Table 17 demonstrates the advantage of using the PG module over the SG module. The primary errors of the PG module in *Intermediate Algebra* and *Prealgebra* are mainly due to inability of the generated python code to express **complex objects (syntax errors)** and **boundary cases**, respectively. In the case of *Geometry* and *Precalculus* problems, a large proportion of errors are caused due to **lack of understanding the plots/figures** (expressed in latex format) accompanying the question. Table 12 presents examples of common syntactical errors (from the MATH dataset) of the PG module, and Table 13 summarizes the different error types of PG+SG setting. Similar to PoT (Chen et al., 2023c), we found the PG module to be less effective for simpler arithmetic problems (removal of Sympy improves the performance by 2-3%). However, the overall performance of SG and PG+SG still remains quite similar. In Table 18, we present an example from GSM-8K where PG+SG commits a reasoning error while the SG setting is correct.

6 Conclusion

We introduce a Tool-augmented Large Language Model (TALM) framework, aka MATHSENSEI, targeted for Mathematical Reasoning. We utilize tools for web-based knowledge retrieval, program generation and execution and symbolic equation solving. We perform extensive ablations over the individual tools, along with varying the order and combination on complex mathematical reasoning datasets (such as MATH). Our best configuration achieves a 13.5% improvement over `gpt-3.5-turbo` (with CoT prompting) on MATH. Our experiments with tool-sequencing methods does not improve over our best configuration. We also observe that benefit of mathematical *TALM*s are minimal for simpler math word problems (in GSM-8k) and its benefit increases as the required complexity and knowledge for the problem increases through AQuA, MMLU-Math.

Limitations

We propose a Tool-Augmented LLM framework (*TALM*), uniquely targeted towards complex mathematical reasoning. Here, we discuss three types of limitations: 1) choice of the set of tools, 2) variants of the PG module for simpler problems and 3) developing mathematical *TALM*-specific planning methods.

1. Here, we choose tools, which intuitively offers knowledge about complex mathematical disciplines and complex equation solving capabilities such as Python with `sympy` library, `WolframAlpha-API` and Bing Web Search API. However, we have not explored other solvers which are targeted towards logical complexity or adding commonsense knowledge. In future, a more universal *TALM* can target adding Z3, SAT solvers and OMCS knowledge base query capabilities.

2. Our Program Generator (PG) module is not only inspired by the program-guided solving methods, but also targetedly use `sympy` library to access complex mathematical equation solving skills. Such skills may not be required for simpler math word problems, as present in GSM-8k. In future, we plan to work on generalizing the PG module so that it is adaptive for simpler problems and focuses mainly on representing the problems in code, only accessing `sympy` capabilities when required.

3. Lastly, we worked on vanilla adaptation of the available planning or tool-sequencing methods directly in the mathematical *TALM* (or MATHSENSEI) context. From our experiments, it is clear that we need to develop more efficient planners that can dynamically choose a sequence of tools based on the problem type (say WA+PG+SG for algebra and PG+CR+SG for Probability), striking a balance between planning beforehand (Plan-And-Solve) and example-wise planning (REACT). We hope our work will inspire researchers to work on such planning methods for mathematical *TALM*s.

Acknowledgements

This work is directly supported by Rakuten India Enterprise Private Limited. Additionally, Debrup Das and PI Somak Aditya are partially supported by the Microsoft Accelerate Foundation Models Research (AFMR) Grant (especially Bing Search API-based experiments have been carried out with the help of AFMR).

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and et al. 2020. Language Models are Few-Shot Learners. In *Proceedings of the 34th Conference on Neural Information Processing Systems*, volume 33 of *NIPS '20*, pages 1877–1901.

Jiaao Chen, Xiaoman Pan, Dian Yu, Kaiqiang Song, Xiaoyang Wang, Dong Yu, and Jianshu Chen. 2023a. Skills-in-Context Prompting: Unlocking Compositionality in Large Language Models.

Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal. 2023b. ReConcile: Round-Table Conference Improves Reasoning via Consensus among Diverse LLMs.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023c. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, and et al. Gehrmann. 2024. PaLM: scaling language modeling with pathways. *J. Mach. Learn. Res.*, 24(1):1–113.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems.

Iddo Drori, Sarah Zhang, Reece Shuttleworth, Leonard Tang, Albert Lu, and et al. 2022. A Neural Network Solves, Explains, and Generates University Math Problems by Program Synthesis and Few-Shot Learning at Human Level. *Proceedings of the National Academy of Sciences*, 119(32):1–10.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate.

Yao Fu, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. 2023. Complexity-Based Prompting for Multi-step Reasoning. In *Proceedings of the 11th International Conference on Learning Representations*, ICLR '23, pages 1–15.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23, pages 10764–10799.

Yiduo Guo, Yaobo Liang, Chenfei Wu, Wenshan Wu, Dongyan Zhao, and Nan Duan. 2023. Learning to Program with Natural Language.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021a. Measuring Massive Multitask Language Understanding. In *Proceedings of the 9th International Conference on Learning Representations*, ICLR '21, pages 1–27.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021b. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the 35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, NIPS '21, pages 1–11.

Jie Huang and Kevin Chen-Chuan Chang. 2023. Towards Reasoning in Large Language Models: A Survey.

Rik Koncel-Kedziorski, Hannaneh Hajishirzi, Ashish Sabharwal, Oren Etzioni, and Siena Dumas Ang. 2015. Parsing Algebraic Word Problems into Equations. *Transactions of the Association for Computational Linguistics*, 3:585–597.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. 2023. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate.

Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL '17', pages 158–167.

Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023a. Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models. In *Proceedings of the 37th Conference on Neural Information Processing Systems*, NIPS '23, pages 1–32.

Pan Lu, Liang Qiu, Wenhao Yu, Sean Welleck, and Kai-Wei Chang. 2023b. A Survey of Deep Learning for Mathematical Reasoning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL '23, pages 14605–14631.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Proceedings of the 37th Conference on Neural Information Processing Systems*, NIPS '23, pages 1–61.

Shen-yun Miao, Chao-Chun Liang, and Keh-Yih Su. 2020. A Diverse Corpus for Evaluating and Developing English Math Word Problem Solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, ACL '20', pages 975–984.

OpenAI. 2023. GPT-4 Technical Report.

Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. 2023. ART: Automatic multi-step reasoning and tool-use for large language models.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Proceedings of the 37th Conference on Neural Information Processing Systems*, NIPS '23, pages 1–13.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, and et al. 2023. Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, pages 1–95.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. 2023. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. In *Findings of the Association for Computational Linguistics*, ACL '23, pages 13003–13051.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: A Large-scale Dataset for Fact Extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, NAACL '18, pages 809–819.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. In *Proceedings of the 36th Conference on Neural Information Processing Systems*, NIPS '22, pages 1–14.

Yuanzhen Xie, Tao Xie, Mingxiong Lin, WenTao Wei, Chenglin Li, Beibei Kong, Lei Chen, Chengxiang Zhuo, Bo Hu, and Zang Li. 2023. OlaGPT: Empowering LLMs With Human-like Problem-Solving Abilities.

Runzhe Yang and Karthik Narasimhan. 2023. The Socratic Method for Self-Discovery in Large Language Models.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering.

In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, EMNLP '18, pages 2369–2380.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Proceedings of the 37th Conference on Neural Information Processing Systems*, NIPS '23, pages 1–14.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models.

Junjie Ye, Xuanting Chen, Nuo Xu, Can Zu, Zekai Shao, Shichun Liu, Yuhan Cui, Zeyang Zhou, Chao Gong, Yang Shen, Jie Zhou, Siming Chen, Tao Gui, Qi Zhang, and Xuanjing Huang. 2023. A Comprehensive Capability Analysis of GPT-3 and GPT-3.5 Series Models.

Chuanyang Zheng, Zhengying Liu, Enze Xie, Zhenguo Li, and Yu Li. 2023. Progressive-Hint Prompting Improves Reasoning in Large Language Models.

Aojun Zhou, Ke Wang, Zimu Lu, Weikang Shi, Sichun Luo, Zipeng Qin, Shaoqing Lu, Anya Jia, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2024. Solving Challenging Math Word Problems Using GPT-4 Code Interpreter with Code-based Self-Verification. In *Proceedings of the 12th International Conference on Learning Representations*, ICLR '24, pages 1–27.

A Planning Experiments

We explore two state-of-the-art planning strategies based following the Chameleon (Lu et al., 2023a) and the REACT (Yao et al., 2022) frameworks and report in in Table 10.

Plan-And-Solve Within the Plan-And-Solve (PAS) framework, a dynamic planner (LLM), generates a plan for a given mathematical problem before the start of execution. In our context, the plan consists of the sequence of modules to be run. Notably, this planning approach is inherently non-adaptive, as the strategy lacks the capability to determine the next module based on feedback and the output of the previously executed modules. To instruct the planner LLM, we provide input prompts containing information about each module, along with few-shot examples representing a possible sequence. The prompts utilized for the planner model are detailed in Table 22.

MATHSENSEI with REACT Planner. The previous modular settings, have a fixed order of execution of the modules. However, we also wish to test out settings where there is power given to the central LLM to call different modules as and when required. This is done by executing (thought, action request, action execution) triplets. The thought serves as a summary of what we have till now in relation to answering the question, the action request is the specific action we wish to take in the next step, and the action execution step calls the necessary module from the modules library to execute the action. An overview of the REACT setting applied to the MATH dataset is presented in Fig. 5. The results for this setting corresponding to each problem type is presented in Table 10.

Results. We evaluate the performance of Plan-And-Solve and REACT on a randomly sampled subset of the MATH dataset of 3100 examples(for which REACT converges). The results show that simple vanilla implementation of the above planners is not sufficient for surpassing our best configuration PG+WA+SG. In particular, the majority of errors for REACT, were as a result of the failure of REACT to converge to a final solution (finish thought state). The variation of the accuracy as a function of the level of the problem (Table 11) shows, REACT* can surpass Plan-And-Solve (PAS) by a small percentage, however it still lags behind our best settings.

Unlike planning in traditional closed world setup datasets such as Blocksworld, Logistics, Depot planning, etc., the task of planning in the mathematical reasoning domain presents multiple differences: Firstly, the set of possible actions is not finite as we can query each tool/module with any input string. Moreover, there are no preconditions that need to be satisfied for executing a particular action which makes the planning space much more unbounded. This can lead to long planning chains with (thought,action,execution) triplets where there may be multiple irrelevant actions. As seen from our work, the strengths and limitations of each tool also varies with the type of datasets, subdomains and difficulty levels, which makes the problem non-trivial. Hence, it turns out to be overwhelming to propose a novel planning strategy in this paper. We plan to explore this issue as a future research direction. A planner with a novel architecture and sufficient mathematical knowledge may be required to tackle this aspect.

	Modules					
Models/APIs	**KR**	**BS**	**WA**	**PG**	**CR**	**SG**
`Bing-Web-Search-API`	✗	✓	✗	✗	✗	✗
`Wolfram-Alpha-API`	✗	✗	✓	✗	✗	✗
`Llama-2-7B`	✓	✗	✗	✗	✗	✗
`Phind-CodeLlama-34B-V2`	✗	✗	✗	✓	✗	✗
`text-davinci-002`	✓	✗	✗	✗	✗	✗
`text-davinci-003`	✓	✓	✓	✗	✗	✓
`gpt-3.5-turbo`	✓	✓	✓	✓	✓	✓

Table 8: Module Inventory.

Setting	**Level 1**	**Level 2**	**Level 3**	**Level 4**	**Level 5**
(SG)	71.8	53.1	41.0	25.6	12.2
(WA+SG)	74.6	60.5	46.6	37.6	21.3
(PG+SG)	83.6	62.4	52.6	40.0	19.8
(WA+PG+SG)	76.4	61.5	54.0	40.2	25.2
(PG+WA+SG)	79.1	62.8	53.9	41.8	26.9
(BS+WA+SG)	74.6	59.3	51.0	35.6	21.0
(WA+BS+SG)	76.0	60.1	52.0	39.9	24.6
(BS+PG+WA+SG)	81.0	60.5	52.9	41.6	25.4

Table 9: Performance of different Settings across varying Levels of Complexity (1-5) on the MATH dataset.

Plan Method	**Alg**	**P.Cal**	**P.Alg**	**Geom**	**Prob**	**N.Th**	**Int.Alg**	**O.Acc**
PAS*	57.3	29.8	65.0	32.4	42.0	47.7	31.9	47.3
REACT*	62.9	30.6	65.1	32.1	42.0	46.1	33.7	48.9
(PG+WA+SG)*	61.4	32.8	65.2	33.4	45.4	54.2	37.6	50.7
(WA+PG+SG)*	64.4	32.1	62.8	32.1	46.9	49.4	38.3	50.6

Table 10: Comparison of planning strategies: Plan-And-Solve (PAS) and REACT with two of our best performing settings on *3072* randomly sampled examples from the MATH dataset (i.e., PG+WA+SG (PG+WA+SG) and WA+PG+SG (WA+PG+SG)). Here X* denotes the use of *3072* samples for evaluating method X.

Setting	**Level 1**	**Level 2**	**Level 3**	**Level 4**	**Level 5**
PAS*	76.0	60.1	53.9	40.5	26.1
REACT*	78.3	62.0	55.4	41.6	27.9
(PG+WA+SG)*	79.3	65.3	54.3	43.9	31.1
(WA+PG+SG)*	78.3	65.6	55.8	43.1	30.1

Table 11: Comparing Performance of different Planning Strategies with two of our Top Performing Settings (i.e., PG+WA+SG (PG+WA+SG) and WA+PG+SG (WA+PG+SG)) by varying Difficulty Level of Problems from the MATH dataset. Here X* denotes the use of *3072* samples for evaluating method X.

Type	**Error message**
Undefined symbols	name 'x' is not defined
Incorrect handling of objects	'FiniteSet' object has no attribute 'subtract'
Undefined functions	name 'divisible_by' is not defined
Use of libraries without import	sympy package not found

Table 12: Syntactic Errors of the PG module.

Dataset	**Subject**	**PG-Exec-Err**	**PG-R-Err**	**SG-Err**	**Egs.**
MATH	Alg	8	5	2	15
	P.Cal	6	9	0	15
	P.Alg	4	11	0	15
	Geom	3	12	0	15
	Prob	8	6	1	15
	N.Th	6	7	3	16
	Int.Alg	14	0	1	15
	O.Cnt	51	48	7	106
GSM-8K	-	1	18	1	20
AQUA	-	1	6	13	20

Table 13: Summary of Error types with PG+SG (PG+SG) setting on a random subset of 106 examples (MATH dataset); for GSM-8K and AQUA we consider 20 random examples, where the setting SG (SG) is correct; **PG-Exec-Err:** Code generated by PG module having syntactical errors; **PG-R-Err:** Executable python code (from PG) having reasoning errors; **SG-Err:** Solution Generator (SG) alters correct output from PG to incorrect; Alg: Algebra, P.Cal: Precalculus, P.Alg: Prealgebra, Geom: Geometry, Prob: Probability, N.Th: Number Theory, Int.Alg: Intermediate Algebra, O.Cnt: Overall Count, Egs.: Examples. Here we report the absolute count of errors across different subjects.

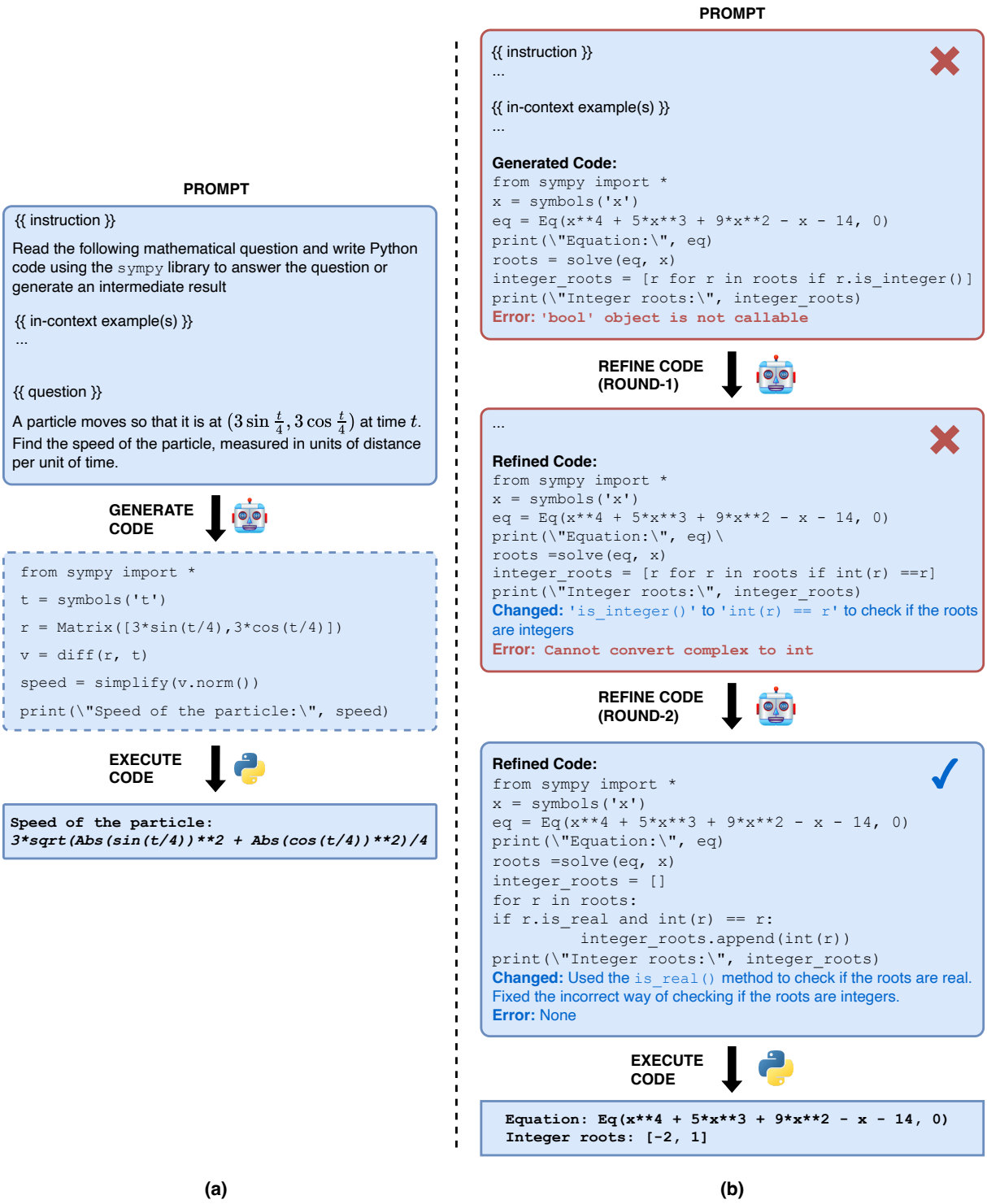

Figure 4: Overview of (a) Python Generator Module and (b) Code Refiner Module

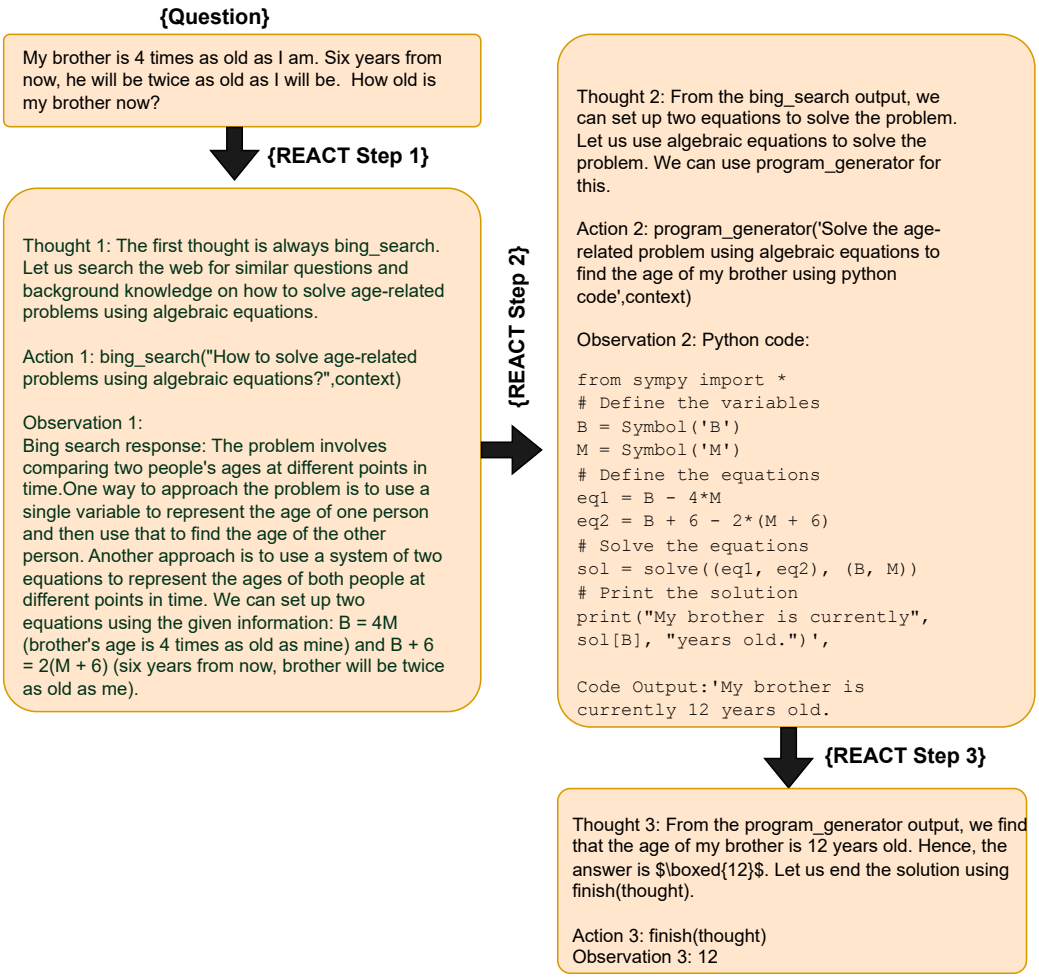

Figure 5: Generated output for example from the MATH dataset for the REACT planning setting.

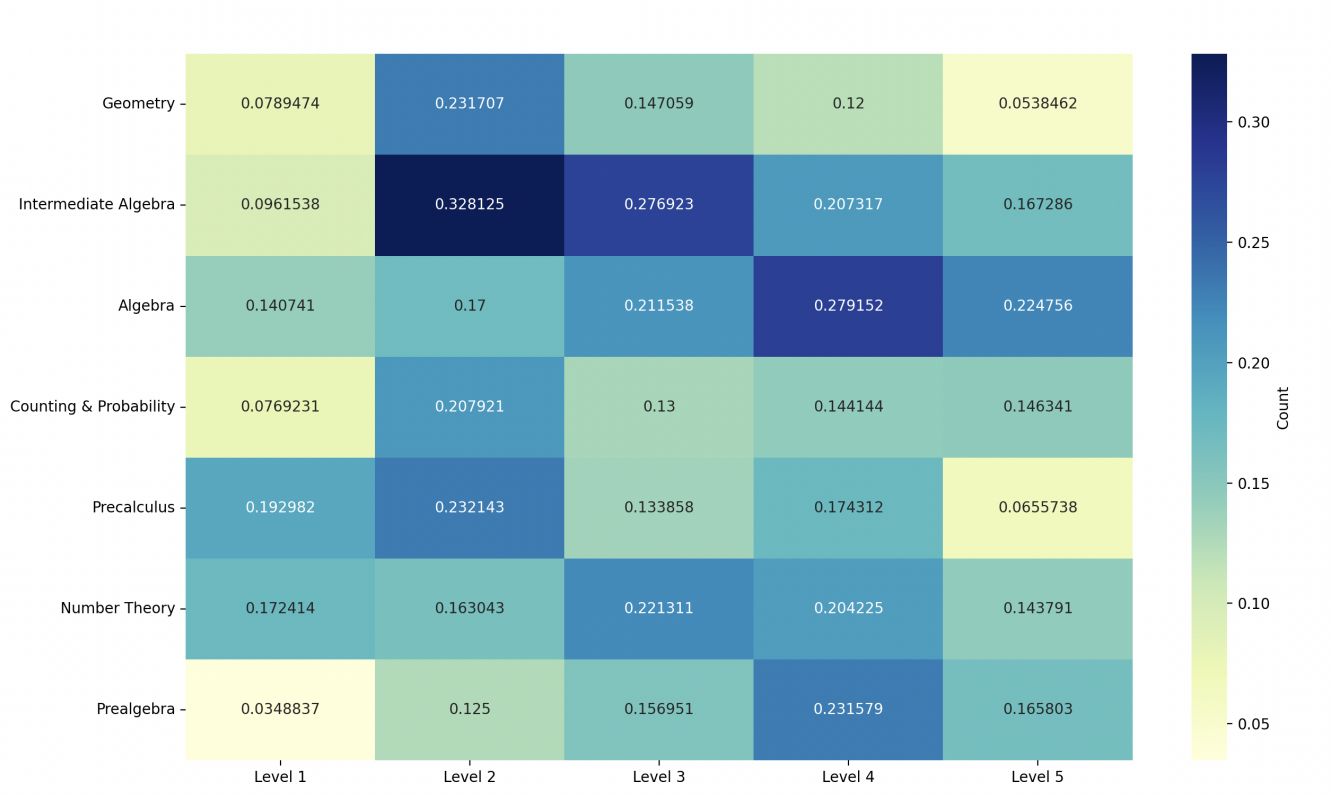

Figure 6: Distribution of examples where WA+SG (+) is correct and SG () is wrong, across problem types and level of difficulty (1-5). The values are normalized by number of examples in each (subject - level) combination.

Setting: WA+SG (✿+✔)		
QUESTION 1: The positive integers up to 2007 are alternately subtracted and added: $[1-2+3-4+\cdots+2001-2002+2003-2004+2005-2006+2007.]$ What is the value of the expression? **GOLD ANSWER:** $\boxed{1004}$	**QUESTION 2:** When the expression $-2x^2-20x-53$ is written in the form $a(x+d)^2+e$, where a, d, and e are constants, what is the sum a+d+e? **GOLD ANSWER:** $\boxed{0}$	**QUESTION 3:** Solve $\frac{2x^2+x+3}{x^2+x+1} = \frac{2x+1}{x+1}$ for x. **GOLD ANSWER:** $\boxed{-2}$
WA API Query: Sum$[(-1)^n * n, \{n, 1, 2007\}]$ [*INCORRECT; since it finds negative of the required answer*] *CORRECT* Query : Sum$[(-1)^{(n+1)} * n, \{n, 1, 2007\}]$	**WA API Query:** CompleteSquare$[-2x^2-20x-53]$ [*CORRECT*]	**WA API Query:** solve $(2x^2+x+3)/(x^2+x+1) = (2x+1)/(x+1)$ for x [*CORRECT*]
WA Output: Yes, the answer is -1004 . [*Logical errors in LLM generated WA API queries*].	**WA Output:** Yes, the answer from this object is: $-2(x+5)^2-3$ [*CORRECT*]	**WA Output:** Yes, the answer is x = -2 [*Since WA module returns a single line answer (in many cases), the downstream SG module fails to provide a step-by-step reasoning.*]
SG Output: We use the Wolfram Alpha API to evaluate the expression Sum$[(-1)^n * n, n, 1, 2007]$. The answer from Wolfram Alpha is -1004. Therefore, the answer is $\boxed{-1004}$ [*INCORRECT*]	**SG Output:** From WolframAlpha response, we find $-2x^2-20x-53 = -2(x+5)^2-3$.Thus we have $a=-2$, $d=-5$, and $e=-3$. So, $a+d+e=-2-5-3=-10$. Therefore, the answer is $\boxed{-10}$ [*INCORRECT; wrong interpretation of correct WA response by the downstream SG module. Value of d should be +5 and not -5.*]	**SG Output** We call the WolframAlpha API to solve the given equation.The answer from Wolfram Alpha is x = -2. Therefore, the answer is $\boxed{-2}$ [*CORRECT; but lacks reasoning chains*]

Table 14: Limitations of the WA Module in WA+SG (✿+✔) setting.

WA Query
$Sum[i^k, \{k, 1, 259\}]$
$Solve[\{725x + 727y == 1500, 729x + 731y == 1508\}, \{x, y\}]$
FactorInteger[3105]
$maximize[y/x, (x-3)^2 + (y-3)^2 == 6]$
period of $cos(bx)$

Table 15: Examples of generated WolframAlpha (WA) Queries.

QUESTION 1: Find the remainder when $x^5 + x^4 + x^3 + x^2 + x$ is divided by $x^3 - 4x$

GOLD ANSWER: $\boxed{5x^2 + 21x}$

QUESTION 2: Let P be the point $(0, 5)$, let Q be the point $(6, 9)$, and let R be the point $(12, 0)$. Determine the area of right-angled $\triangle PQR$.

GOLD ANSWER: $\boxed{39}$

Setting: SG

Response:
The result of the division is $x^2 + 5x + \frac{5x}{x^3 - 4x}$

[*INCORRECT; Calculation error in intermediate step*]

Response:
$QR = \sqrt{(12-6)^2 + (0-9)^2} = \sqrt{180}$

[*INCORRECT; Error in distance calculation in intermediate step.*]

Setting: WA+SG

WA API Query:
PolynomialRemainder[$x^5 + x^4 + x^3 + x^2 + x, x^3 - 4x, x$]
[CORRECT]

WA Output:
Yes, the answer is $5x^2 + 21x$
[*CORRECT*]

SG Output:
The answer from Wolfram Alpha is found to be $5x^2 + 21x$. Therefore, the answer is $\boxed{5x^2 + 21x}$
[*CORRECT*]

WA API Query:
triangle((0,5),(6,9),(12,0))
[*CORRECT*]

WA Output:
Yes, the answer is: Triangle shape: right triangle; Edge lengths: $(2\sqrt{13}, 3\sqrt{13}, 13) \approx (7.211, 10.816, 13)$; - Area: 39; Perimeter: $13 + 5\sqrt{13} \approx 31.027$; Interior angles: $cos^{-1}(3/\sqrt{13})$ rad; $cos^{-1}(2/\sqrt{1})$ rad; $\pi/2$ rad $\approx$ (0.588 rad, 0.982 rad, 1.570 rad); Interior angle sum: $180° = \pi$ rad ≈ 3.142 rad.
[*CORRECT*]

SG Output:
We call the Wolfram Alpha API to find the area of the triangle formed by the points $(0, 5)$, $(6, 9)$, and $(12, 0)$; The answer from Wolfram Alpha is found to be 39; Therefore, the answer is $\boxed{39}$
[*CORRECT*]

Table 16: Utility of the WA module in WA+SG (+) setting.

QUESTION: The average of Amys, Ben, and Chris's ages is 6. Four years ago, Chris was the same age as Amy is now. In four years, Ben's age will be 35 of Amy's age at that time. How many years old is Chris now?

GOLD ANSWER: $\boxed{10}$

Setting: SG (✅)

Response:

$\frac{1}{3}(a+b+c) = 6; \quad c-4 = a; \quad b+4 = \frac{3}{5}(a+4).$

Simplifying the third equation, we get $5b + 20 = 3a + 12$. Substituting $c = a + 4$ into the first equation and simplifying, we get $a + b = 10$.

[*INCORRECT; Correct equation: 2a +b =14*]

Setting: PG+SG (🐍+✅)

Response:

```
from sympy import *
a, b, c = symbols('a b c')
eq1 = Eq((a+b+c)/3, 6)
eq2 = Eq(c-4, a)
eq3 = Eq(b+4, Rational(3,5)*(a+4))
sol = solve((eq1, eq2, eq3), (a, b, c))
print("Chris's age is", sol[c], "years old")
```

[Leverages the classes **Rational**, **Eq** and method **solve** from Sympy]

PG Output: Chris's age is **10** years old

SG Output: Let Amy's age be a, Ben's age be b, and Chris's age be c. The average of their ages is $\frac{a+b+c}{3} = 6$, Four years ago, Chris's age was c-4, which was the same as Amy's age at that time, so $c - 4 = a$, In four years, Ben's age will be $b + 4$, which will be $\frac{3}{5}$ of Amy's age at that time, so $b + 4 = \frac{3}{5}(a + 4)$, Solving the system of equations gives $a = 6$, $b = 2$, and $c = 10$. Therefore, the answer is $\boxed{10}$ years old.

[*CORRECT*]

Table 17: Utility of the PG module in PG+SG (🐍+✅) setting.

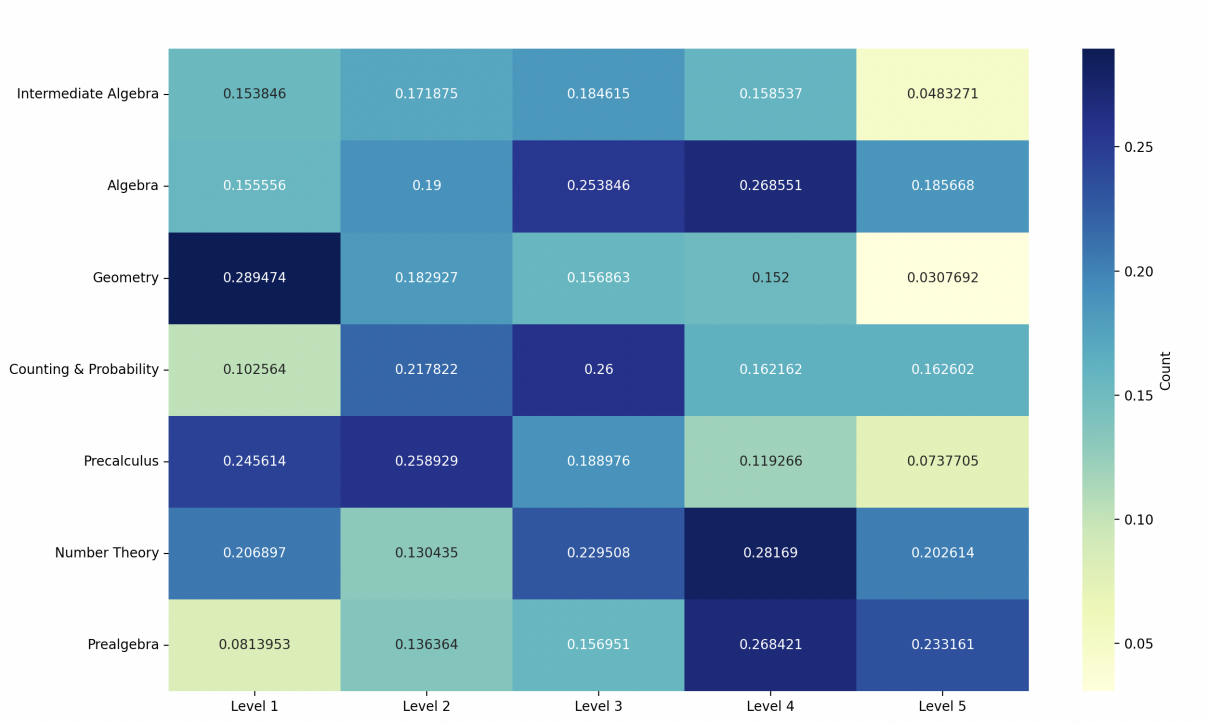

Figure 7: Distribution of examples where PG+SG (+) is correct and SG ( ) is wrong, across problem types and level of difficulty (1-5). The values are normalized by number of examples in each (subject - level) combination.

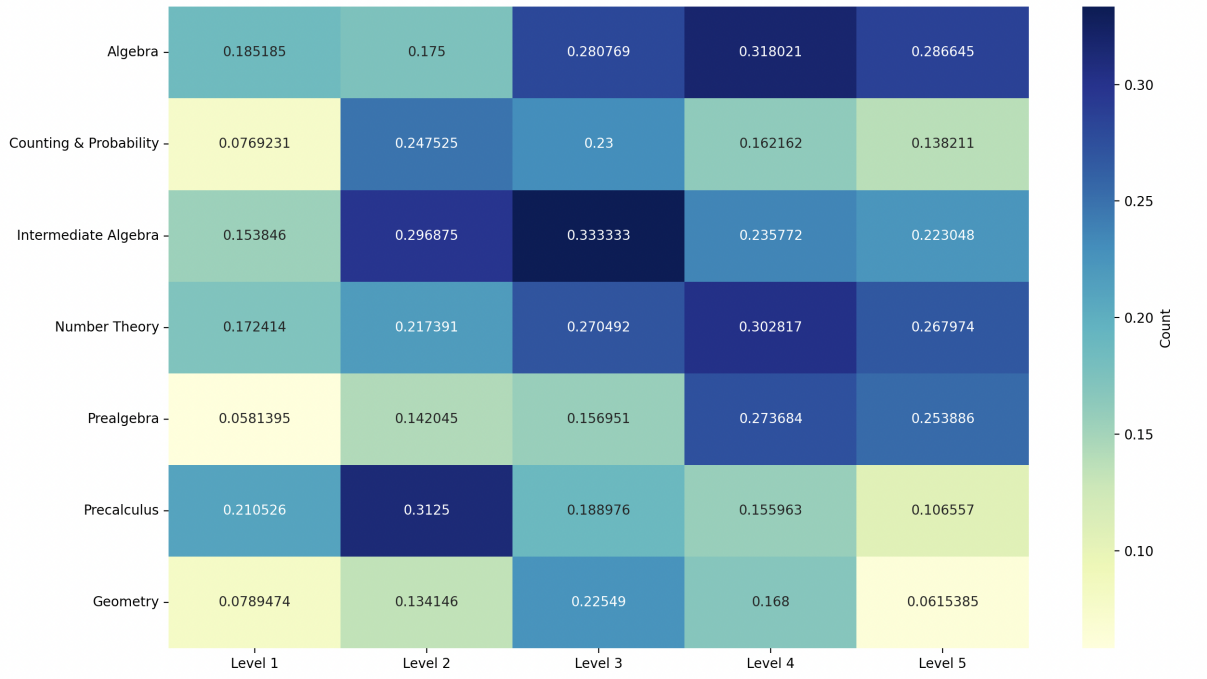

Figure 8: Distribution of examples where PG+WA+SG (+ +) is correct and SG ( ) is wrong, across problem types and level of difficulty (1-5). The values are normalized by number of examples in each (subject - level) combination.

QUESTION: Uriah's book bag is getting too heavy for him. He needs to remove 15 pounds from it. His comic books weigh 1/4 pound each and his toys weigh 1/2 pound each. If he removes 30 comic books, how many toys does he need to remove?

GOLD ANSWER: 30 comic books weigh 7.5 pounds because 30 x .25 = «30*.25=7.5»7.5 He needs to remove 7.5 more pounds because 15 - 7.5 = 7.5 He needs to remove 15 toys because 7.5 / .5 = 15

Setting: SG (✓)

Response:

We know that Uriah needs to remove a total of 15 pounds from his bag. He removes 30 comic books, which weigh a total of 30*(1/4) = 7.5 pounds. So he still needs to remove 15 - 7.5 = 7.5 pounds.Let the number of toys he needs to remove be x.The weight of x toys is x*(1/2) = 0.5x pounds. We need to solve the equation 0.5x = 7.5 to find x. Solving for x, we get x = 15. Therefore, Uriah needs to remove 15 toys from his bag. The answer is 15. [*CORRECT*]

Setting: PG+SG (🐍+✓)

Response:

```
from sympy import *
x = symbols('x')
eq = Eq((1/4)*(30-x) + (1/2)*x, 15)        [Wrong equation - reasoning error]
print("Equation:",eq)
sol = solve(eq, x)
print("Solution x:",sol)
ans = 30 - sol[0]
print("Toys to remove:",ans)
```

PG Output: Equation: Eq(0.25*x + 7.5, 15); Solution x: [30.0000000000000]; Toys to remove: 0;

SG Output: From Python code we get that Uriah needs to remove 0 toys. The answer is 0. [*INCORRECT*]

Table 18: Demonstration of a Reasoning Error committed by the PG module in PG+SG (🐍+✓) setting in a GSM-8K problem.

INSTRUCTION

Read the following question, generate the background knowledge or relevant mathematical information as the context information that could be helpful for answering the question.

IN-CONTEXT EXAMPLES

Question: The smallest distance between the origin and a point on the graph of [$y = \frac{1}{2}x^2 - 9$] can be expressed as a. Find a^2.
Modules used till now: []
Mathematics Problem Type: Algebra
Level of Problem: Level 5
Knowledge:
- The question involves the concept of distance between two points in a coordinate system.
- The distance between the origin and a point (x, y) on the graph is given by the formula $\sqrt{(x^2 + y^2)}$.
- Minimizing $(x^2 + y^2)$ is same as minimizing $\sqrt{(x^2 + y^2)}$.
- Substitute $y = 1/2x^2 - 9$ into the equation to get $x^2 + (1/2x^2 - 9)^2$.
- Take derivative of above equation and set it to 0.

... **[More few shot examples]**

INPUT QUESTION

Question: What is the following value when expressed as a common fraction: $\frac{1}{2^1} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^8} + \frac{1}{2^9} + \frac{1}{2^{10}}$?

Mathematics Problem Type: Algebra
Level of Problem: Level 4

OUTPUT

Knowledge:
- The question involves understanding of geometric series in mathematics.
- A geometric series is a series of numbers where each term is obtained by multiplying the previous term by a fixed constant, called the common ratio.
- The sum of the first n terms of a geometric series is given by the formula $S_n = a(1 - r^n)/(1 - r)$, where a is the first term and r is the common ratio.
- In this case, the first term is 1/2 and the common ratio is also 1/2.
- We need to find the sum of the first 10 terms of the series, which can be calculated using the above formula.

Table 19: LLM-based Knowledge Retrieval Prompt

INSTRUCTION

Read the following question to generate thought, Query for searching Bing Web Search API, that will help to solve the entire problem or specific subproblems in the question.

IN-CONTEXT EXAMPLES

Question: When the expression $-2x^2 - 20x - 53$ is written in the form $a(x + d)^2 + e$, where a, d, and e are constants, what is the sum $a + d + e$?
Modules used till now: []
Mathematics Problem Type: Algebra
Level of Problem: Level 5
Thought: Since the question involves completing the square let us search how to complete the square.
Query: How do we complete the square of a quadratic equation?

Question: A circle is circumscribed about an equilateral triangle with side lengths of 6 units each. What is the area of the circle, in square units? Express your answer in terms of π.
Modules used till now: []
Mathematics Problem Type: Geometry
Level of Problem: Level 5
Thought: To solve this problem, we can search how to find the circumradius of an equilateral triangle.
Query: How to find the circumradius of an equilateral triangle?

... **[More few shot examples]**

INPUT QUESTION

Question: The point $P = (1, 2, 3)$ is reflected in the xy-plane, then its image Q is rotated by 180° about the x-axis to produce R, and finally, R is translated by 5 units in the positive-y direction to produce S. What are the coordinates of S?
Mathematics Problem Type: Precalculus
Level of Problem: Level 3

OUTPUT

Thought: To solve this problem, we can search the web for formulas related to reflection, rotation and translation.
Query: What are the formulas for reflection, rotation, and translation in 3D space?

Table 20: Bing Web Search Query generator Prompt for Concepts Search

INSTRUCTION

Read the following question to generate thought, answer and final query for searching Wolfram Alpha API, that is being used to solve the problem or specific subproblems.

IN-CONTEXT EXAMPLES

Question: When the expression $-2x^2 - 20x - 53$ is written in the form $a(x + d)^2 + e$, where a, d, and e are constants, what is the sum $a + d + e$?
Modules used till now: []
Mathematics Problem Type: Algebra
Level of Problem: Level 5
Thought: To find the answer to the problem, we need to rewrite the quadratic expression $-2x^2 - 20x - 53$ in the form $a(x + d)^2 + e$, where a, d, and e are constants. Then, we find the sum a+d+e. Do you know the Wolfram Alpha command to do this?
Answer: Yes, the Wolfram Alpha command to factor the given quadratic expression is: factor$(-2x^2 - 20x - 53)$
Final Query: factor$(-2x^2 - 20x - 53)$

Question: Harold tosses a nickel four times. What is the probability that he gets at least as many heads as tails?
Modules used till now: []
Mathematics Problem Type: Counting and Probability
Level of Problem: Level 5
Thought: To find the probability that Harold gets at least as many heads as tails when tossing a nickel four times, we can use combinatorial techniques to count the favorable outcomes. However, to simplify the calculations, we can leverage Wolfram Alpha to compute the probability efficiently. Do you know the Wolfram Alpha command to find the probability for this problem?
Answer: Yes, the Wolfram Alpha command to find the probability of getting at least as many heads as tails when tossing a nickel four times is: $Binomial[4, k] * (1/2)^4$, where k ranges from 2 to 4 (inclusive).
Final Query: $Sum[Binomial[4, k] * (1/2)^4, k, 2, 4]$

... **[More few shot examples]**

INPUT QUESTION

Question: Find the constant c such that $(x^2 - 4x + 3)(x + 5) - (x^2 + 4x - 5)(x - c) = 0$ for all x.

OUTPUT

Thought: To find the constant c such that the given expression is equal to zero for all x, we can expand the expression and collect like terms. Do you know the Wolfram Alpha command to do this?

Answer: Yes, the Wolfram Alpha command to expand the given expression and set it equal to zero is: expand $((x^2 - 4x + 3)(x + 5) - (x^2 + 4x - 5)(x - c)) = 0$

Final Query: expand $((x^2 - 4x + 3)(x + 5) - (x^2 + 4x - 5)(x - c)) = 0$

Table 21: Wolfram Alpha API Query generator Prompt

INSTRUCTION

You need to act as a policy model, that given a question, determines the sequence of modules that can be executed sequentially to solve the question.

The modules are defined as follows:
- wolfram-alpha-search: Calls the Wolfram-Alpha API to solve the given question or intermediate steps in the question.
- bing-search: Retrieves similar questions, background knowledge, useful information using the Bing Web Search API.
- python-generator: This module generates an executable python program that can solve the given question using the Sympy library.
- solution-generator: This module generates a detailed solution to the question based on the already generated context. Normally, solution-generator will incorporate the information from wolfram-alpha-search, bing-search, python-generator. It is always the last module to be executed.

IN-CONTEXT EXAMPLES

Question: Determine the number of ways to arrange the letters of the word ELEVEN.
Modules: ['bing-search','solution-generator']

INPUT QUESTION

Question: If the numbers 4, 5 and 6 are each used exactly once to replace the letters in the expression $A(B - C)$, what is the least possible result?

OUTPUT

Modules: ['python-generator', 'solution-generator']

Table 22: Example of Planner Prompt and Output in Plan-And-Solve (PAS).

Resource	**URL**
open-source icons	https://iconduck.com/icons/
llama-2 icon	https://llama-2.ai/wp-content/uploads/2023/08/Llama-2-icon-150x150.png
codellama icon	https://codellama.dev/icons/black-transparentbg.png
python icon	https://s3.dualstack.us-east-2.amazonaws.com/pythondotorg-assets/media/community/logos/python-logo-only.png
azure openai service	https://azure.microsoft.com/en-us/products/ai-services/openai-service/
bing web search api service	https://www.microsoft.com/en-us/bing/apis/bing-web-search-api

Table 23: Online Resources